

Encorely

It's the beat, don't go alone.

Emmanuel Dávila

Juan Diego Calle

Camilo Rincón

Universidad Tecnológica de Antioquia

Medellín, Colombia mayo, 2026

1. Definición del proyecto

1.1. Título del proyecto

It's the beat, don't go alone. El backend Encorely es el motor de servicios para esta aplicación social orientada a conciertos y festivales, que modela el "ADN musical" de cada persona a partir de interacciones tipo swipe sobre pistas de audio y calcula afinidad entre usuarios para habilitar radar de compatibilidad, conversaciones y salas grupales asociadas a un evento.

1.2. Descripción general del problema a resolver

En eventos en vivo es difícil identificar de forma rápida y objetiva a personas con gustos musicales compatibles. Las redes sociales genéricas no utilizan señales auditivas ni comportamiento de escucha; Encorely propone inferir preferencias reales mediante swipes sobre tracks y características de audio (energía, bailabilidad, valencia, tempo), aplicar un umbral mínimo de interacciones para estabilizar el perfil y ofrecer compatibilidad cuantificada (distancia euclidiana / umbral de afinidad) antes de exponer contacto social sensible como chat o radar por evento.

1.3. Público objetivo

- Personas que asisten a conciertos o festivales y desean conocer a otras personas con afinidad musical verificada.
- Usuarios que ya utilizan Spotify u otros proveedores integrados para autenticación y enriquecimiento de datos.
- Equipos de producto que integran el backend mediante REST y canales en tiempo real (SignalR).

1.4. Alcance del proyecto

Incluye:

- API REST en ASP.NET Core (.NET 9) con documentación Swagger en la ruta /docs.
- Persistencia en PostgreSQL mediante Dapper; modelo de usuarios, swipes, perfiles musicales, matches, mensajes y venue.

- Arquitectura orientada a eventos con Apache Kafka y worker asíncrono (EncorelyWorker) para procesar swipes.
- Autenticación JWT y login multi-proveedor (Spotify, Google, correo).
- SignalR: hub de notificaciones y hub de salas de venue.
- Integración con Spotify para tracks y generación de playlist "DNA mix" entre dos usuarios.
- Caché Redis configurada en la API.

No incluye (fuera de alcance de este repositorio backend):

- Aplicación cliente completa (interfaz móvil o web); solo se define la superficie de API y tiempo real.
- Gestión comercial de venta de entradas o pagos.
- Moderación avanzada basada en IA (existe moderación básica por palabras clave en venue).
- Garantías legales de uso de datos de terceros más allá de lo implementado en integraciones.

2. Análisis de requerimientos

2.1. Requerimientos funcionales

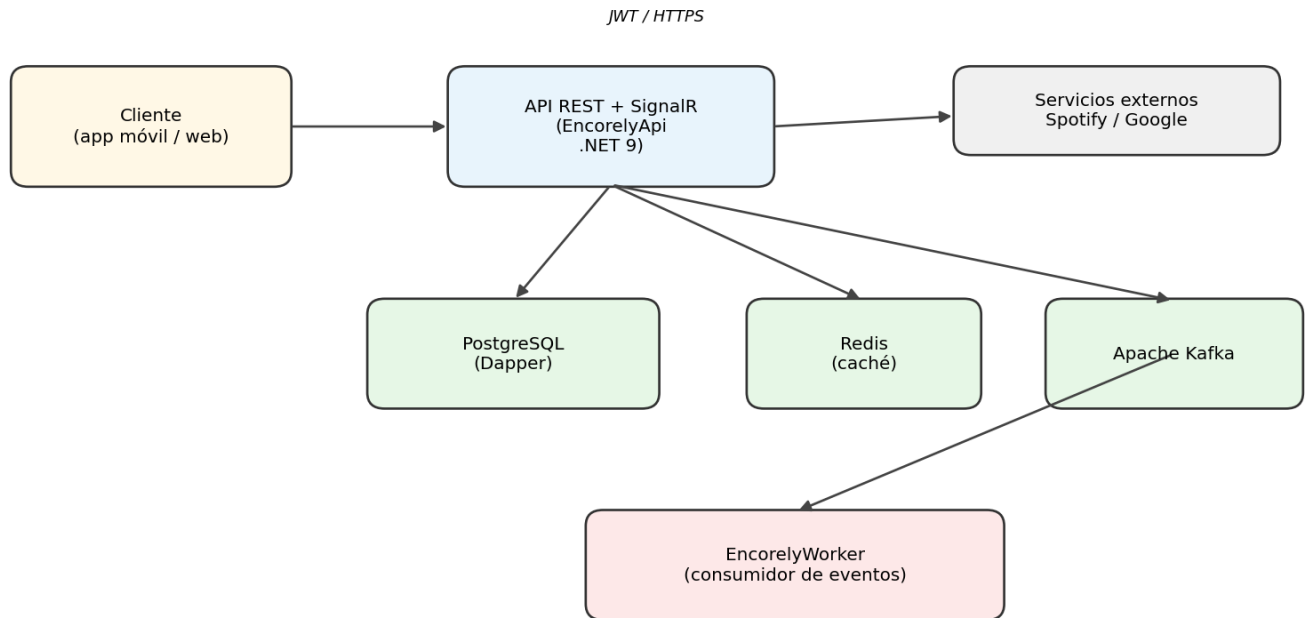
- Autenticación: login/registro con Spotify, Google y correo electrónico; emisión de respuestas coherentes con IdentityService.
- Gestión de perfil: consultar datos del usuario y actualizar estado de ánimo (Moshpit, Chill, VIP).
- Swipes: obtener siguiente track sugerido y registrar swipe (like/dislike/superlike) disparando procesamiento asíncrono.
- Eventos: listar feed de eventos cercanos y obtener candidatos compatibles para un evento con filtro de mood opcional.
- Umbral social: el radar de matches por evento exige al menos 25 swipes registrados.
- Matches: listar pendientes, aceptar match y obtener identificador de sala de chat.
- Chat 1-1: leer historial y enviar mensajes asociados a un match (con validaciones según proveedor).
- Venue: crear sala temporal por evento, listar y publicar mensajes; moderación manual vía endpoint DELETE.
- Playlist compartida: generar mezcla DNA entre dos usuarios con tokens de Spotify.
- Tiempo real: notificaciones de match y ADN completado; difusión de mensajes de venue por SignalR.

2.2. Requerimientos no funcionales (deseables)

- Seguridad: JWT bearer, HTTPS en despliegue, variables de entorno para secretos y cadena de base de datos.
- Rendimiento y escalabilidad: desacoplamiento vía Kafka para cómputo pesado de perfil musical; Redis para caché.
- Disponibilidad: posibilidad de desplegar en contenedores (Docker Compose documentado).
- Usabilidad del contrato API: Swagger UI en /docs; nombres camelCase en JSON.
- Mantenibilidad: capas Domain, Application, Infrastructure, Api y Worker.
- Observabilidad (mejora sugerida en el proyecto): health checks y dead-letter para Kafka.
- Privacidad: minimizar datos en logs; moderación de contenido grupal básica.

3. Diseño de la solución

3.1. Arquitectura general

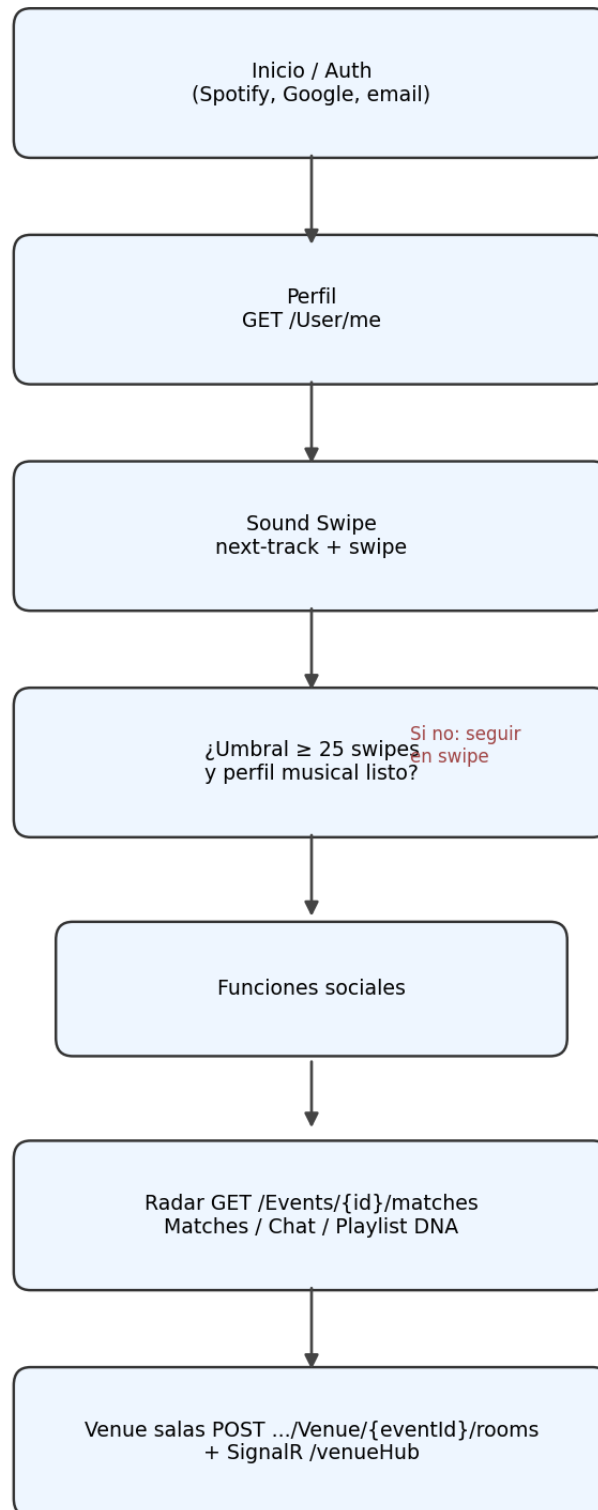


Eventos de swipe → Kafka → refinamiento del perfil musical

Fuente: elaboración propia a partir del código del repositorio.

La aplicación cliente consume REST y WebSockets (SignalR). La API orquesta casos de uso, persiste en PostgreSQL y publica eventos a Kafka; el worker consume dichos eventos para actualizar el perfil musical. Redis actúa como caché distribuida según la configuración en Program.cs.

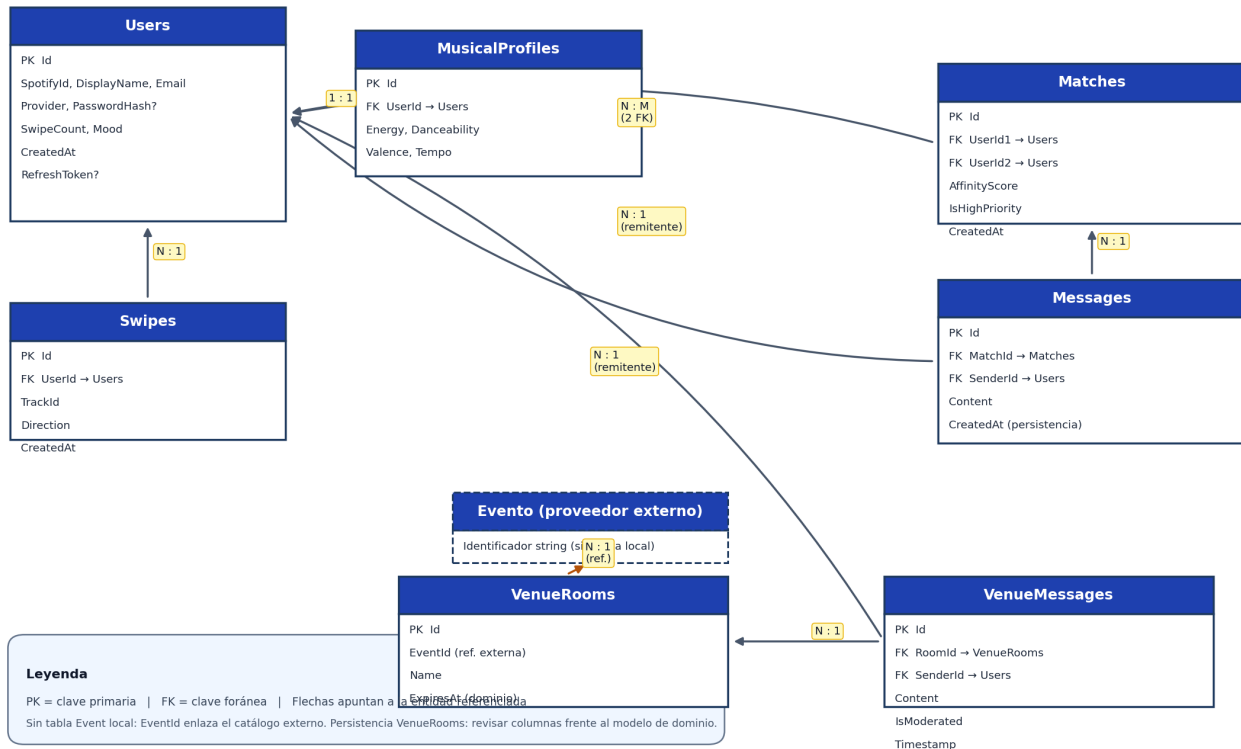
3.2. Diagrama de flujo de navegación de la app



Flujo simplificado: autenticación, construcción del perfil mediante swipes, comprobación del umbral de 25 interacciones y habilitación de funciones sociales (radar, chat, venue, playlist).

3.3. Modelo entidad-relación de la base de datos

Modelo entidad-relación — Encorely (PostgreSQL, esquema referido por repositorios Dapper)



Fuente: EncorelyModels, INSERT en EncorelyRepository y contratos de dominio.

El diagrama muestra cada tabla con atributos PK/FK, cardinalidades en las relaciones y el evento musical como entidad externa (sin fila local). Las flechas siguen la dirección referencial habitual (hacia la entidad referenciada). El esquema físico en PostgreSQL nombra tablas entre comillas (por ejemplo "Users", "Swipes") y las FK se materializan mediante los campos indicados en los INSERT de los repositorios Dapper.

3.4. Especificación de endpoints de la API

POST /api/v1/Auth/spotify

Cuerpo JSON: token (string). Login con token de Spotify; respuesta 202 Accepted.

POST /api/v1/Auth/google

Cuerpo JSON: token (string). Login con Google Identity Token; 202 Accepted.

POST /api/v1/Auth/register

Cuerpo: email, password. Registro por correo; 202 Accepted.

POST /api/v1/Auth/login

Cuerpo: email, password. Login por correo; 200 OK.

GET /api/v1/User/me

Query obligatorio: userId (GUID). Perfil, proveedor, swipeCount y mood.

PUT /api/v1/User/settings

Cuerpo: userId, mood (enum de concierto). Actualización de preferencia; 204 No Content.

GET /api/v1/Swipes/next-track

Query: userId. Siguiente track para evaluar.

POST /api/v1/Swipes/interactions/swipe

Cuerpo: userId, trackId, direction (enum). Registra swipe; 202 Accepted.

GET /api/v1/Events/feed

Sin parámetros. Lista de eventos cercanos.

GET /api/v1/Events/{eventId}/matches

Query: userId; opcional targetMood. Radar de compatibles; 403 si swipeCount < 25.

GET /api/v1/Chat/{roomId}/messages

Query: userId. Historial de mensajes del match.

POST /api/v1/Chat/{matchId}/messages

Query: userId; cuerpo: texto plano. Envío de mensaje; respuesta con id y marca temporal.

GET /api/v1/Matches/pending

Query: userId. Matches pendientes de aceptación.

POST /api/v1/Matches/{matchId}/accept

Query: userId. Aceptar match; cuerpo de respuesta incluye roomId.

POST /api/v1/Venue/{eventId}/rooms

Query: name; opcional durationHours (por defecto 4). Crea sala temporal ligada al evento.

GET /api/v1/Venue/{roomId}/messages

Sin query obligatorio. Mensajes no moderados de la sala.

POST /api/v1/Venue/{roomId}/messages

Query: userId; cuerpo: texto. Publicar mensaje; puede moderarse por palabras clave.

DELETE /api/v1/Venue/messages/{messageId}

Query opcional: reason. Moderación manual de un mensaje.

POST /api/v1/Playlist/dna-mix

Query: userId1, userId2, accessToken1, accessToken2. Genera playlist compartida tipo DNA mix.

SignalR (WebSockets)

- /notificationHub — grupos por usuario; eventos MatchFound, DnaCompleted.
- /venueHub — salas de venue; métodos JoinVenueRoom, LeaveVenueRoom, SendVenueMessage; eventos ReceiveVenueMessage, UserJoined, UserLeft.

Referencias

Equipo Encorely. (2026). Encorely backend [Software]. Repositorio del proyecto.
Documentación interactiva: Swagger UI en /docs cuando la API está en ejecución.